

FINAL PROJECT DOCUMENTATION

Secure Credential & Key Management System for IoT Devices

Course: IoT Security and Embedded Systems

Semester: Fall 2026

Submission Date: September 2026

Team Members

- Project team member 1 — ID not provided
- Project team member 2 — ID not provided
- Project team member 3 — ID not provided
- Project team member 4 — ID not provided

Instructor: Not specified in the source documentation

2. Abstract

This project presents a secure credential and cryptographic-key management system for Internet of Things (IoT) devices. The system addresses the risks of hardcoded credentials, plaintext storage, shared keys, database theft, man-in-the-middle attacks, replay attacks, and brute-force attempts. A centralized Python security server generates unique credentials for each device, encrypts sensitive values using AES-256, and delivers them through a one-time HTTPS provisioning workflow. The ESP32 device stores the provisioned values in encrypted Non-Volatile Storage (NVS) and communicates with a Mosquitto MQTT broker through TLS 1.2 or higher. The server provides authentication, authorization, key rotation and revocation, audit logging, and a web dashboard for monitoring and administration. The implementation uses ESP32-WROOM-32 hardware, Python and Flask services, SQLite, Mosquitto, Bootstrap, Chart.js, and the paho-mqtt client. The documented implementation contains approximately 6,500 lines of code, more than 30 files, 11 database tables, four active services, ten dashboard pages, 28 successful tests, and a reported security score of 9.5/10. The resulting architecture provides per-device secrets, encrypted storage, controlled provisioning, and centralized lifecycle management.

3. Introduction

3.1 Background and Motivation

IoT deployments combine constrained embedded devices, wireless networks, brokers, backend services, and administrative interfaces. These components frequently handle passwords, MQTT credentials, certificates, encryption keys, and sensor data. If these secrets are placed directly in firmware or stored without protection, extracting one device or database can expose an entire deployment. The project was motivated by the need to replace ad-hoc secret handling with a centralized security lifecycle that supports provisioning, storage, monitoring, rotation, and revocation.

3.2 Problem Statement

The initial approach stores sensitive values directly in source code or in exposed storage. A firmware dump can therefore reveal credentials; a stolen database can reveal all device secrets; a single shared key creates a system-wide compromise; and the absence of rotation or revocation prevents recovery after leakage. The project solves this gap by issuing unique credentials per device, encrypting values at rest and in transit, and providing administrative control over the complete credential lifecycle.

3.3 Project Objectives

- Design a centralized credential and key-management service for IoT devices.
- Provide one-time HTTPS provisioning with device-specific secrets.
- Protect stored credentials using AES-256 and encrypted ESP32 NVS.
- Secure MQTT communication with TLS 1.2+ and authenticated sessions.
- Support key rotation, credential revocation, audit logging, and monitoring.
- Validate the implementation through unit, integration, security, and operational tests.

3.4 Scope and Limitations

The scope covers ESP32 devices, encrypted NVS, HTTPS provisioning, MQTT over TLS, a Python security server, SQLite persistence, a dashboard, and administrative key operations. The documented implementation does not claim a hardware secure element, formal cryptographic certification, or unlimited fleet-scale performance testing. The project is also constrained by embedded memory and processing limits, development time, the availability of test devices, and the need to maintain compatibility with common IoT tooling.

4. System Overview

4.1 High-Level Description

At startup, an ESP32 device checks whether it has been provisioned. If not, it contacts the

provisioning API over HTTPS. The server validates the device identity, generates unique credentials and keys, encrypts the response, and records the event. The device stores the values in encrypted NVS. After provisioning, the device connects to the MQTT broker using TLS, publishes sensor data, and participates in an authenticated session. Administrators use the dashboard or command-line tools to inspect devices, suspend or revoke credentials, rotate keys, and review audit records.

4.2 Functional Description

- Device provisioning and identity validation.
- Unique credential and key generation.
- AES-256 encryption of secrets and sensitive database values.
- Encrypted NVS storage on ESP32 devices.
- MQTT messaging through a TLS-enabled Mosquitto broker.
- Authentication, authorization, session management, and audit logging.
- Dashboard monitoring, device management, and key lifecycle operations.

4.3 Use-Case Scenario

When a new ESP32 device is powered on, it requests provisioning through the HTTPS API. The server verifies the device identity and generates a unique credential package. The package is encrypted and delivered once. The device saves it in encrypted NVS, establishes a TLS-protected MQTT connection, and publishes sensor data. An administrator can later view the device in the dashboard, rotate its key, suspend it, or revoke it. The next device connection is then required to use the new authorization state.

5. Hardware Design

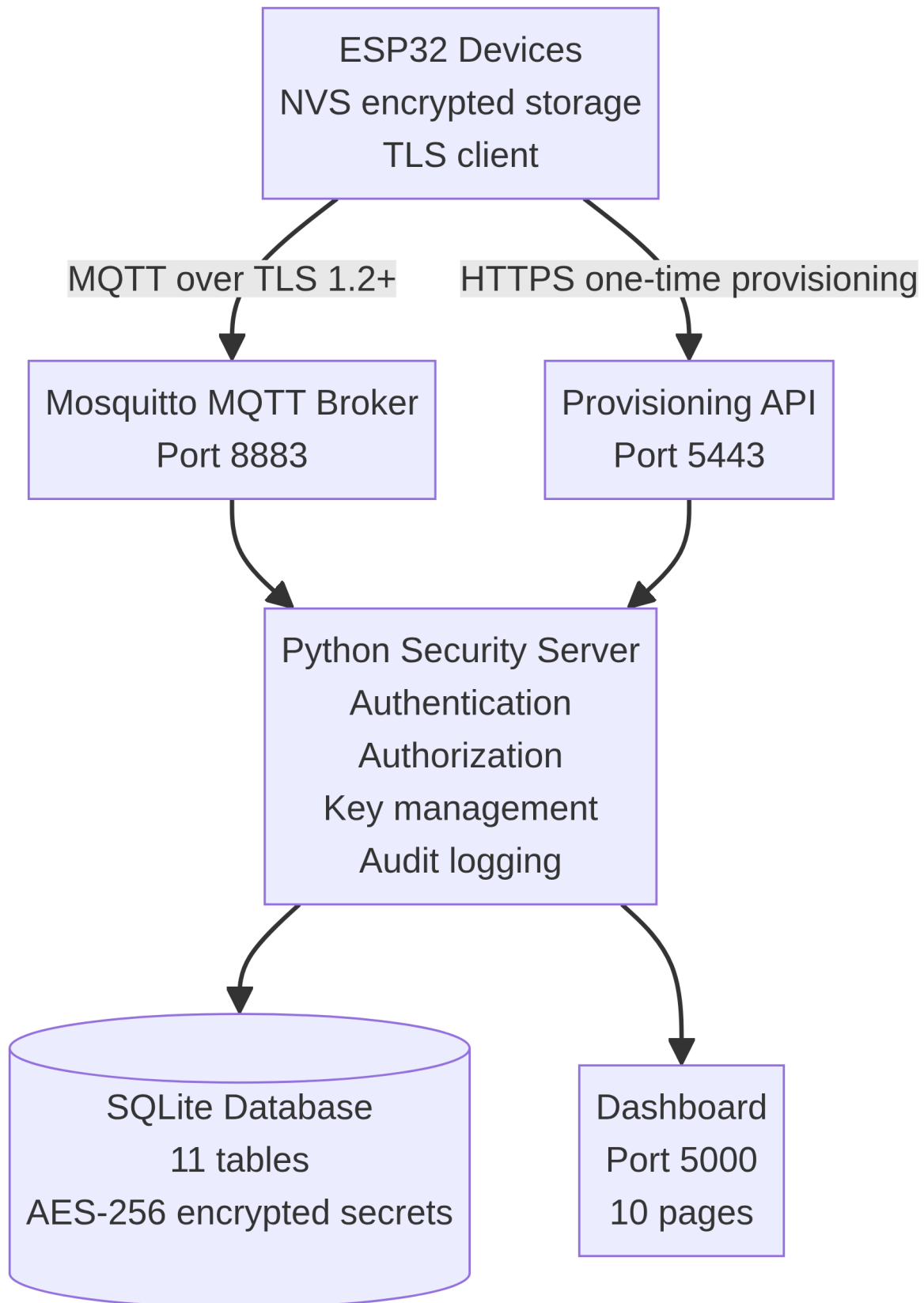
5.1 Components List

Component Name	Specification / Model	Quantity
Microcontroller	ESP32-WROOM-32 with built-in sensors / Wi-Fi	Multiple
MQTT broker host	Computer or server running Mosquitto 2.x	1
Security server host	Python 3.14 / Flask 3.x service	1
Storage	SQLite 3.x database with 11 tables	1
Network	Wi-Fi 2.4 GHz; TLS 1.2+	1 network
Administration interface	Dashboard with Bootstrap 5.3.2 and Chart.js 4.4.0	1

5.2 Component Justification

ESP32-WROOM-32 provides an affordable Wi-Fi-enabled embedded platform with NVS support and sufficient resources for TLS and MQTT. Mosquitto is lightweight, standards-compatible, and supports TLS listeners. Python and Flask enable rapid development of authentication, provisioning, and management APIs. SQLite reduces deployment complexity for a compact project while still supporting the required relational records. Bootstrap and Chart.js provide a practical dashboard without introducing a heavy frontend framework.

5.3 Block Diagram



5.4 Circuit Diagram

The source documentation does not provide a pin-level circuit schematic. The hardware design is primarily a networked ESP32 system; therefore, the relevant connection is the ESP32 power and sensor configuration together with the Wi-Fi/TLS communication path. A detailed electrical schematic should be added if external sensors or actuators are connected in a future hardware revision.

5.5 Power Considerations

The ESP32 is treated as a 3.3 V logic platform and may be powered through USB or a regulated supply. The broker, security server, database, and dashboard run on a host computer or server. Actual current consumption depends on Wi-Fi transmit activity, sensor duty cycle, TLS handshakes, and sleep configuration; it should be measured with a multimeter for a production deployment. No 5 V signal should be connected directly to ESP32 GPIO pins without appropriate level shifting.

6. Software Design

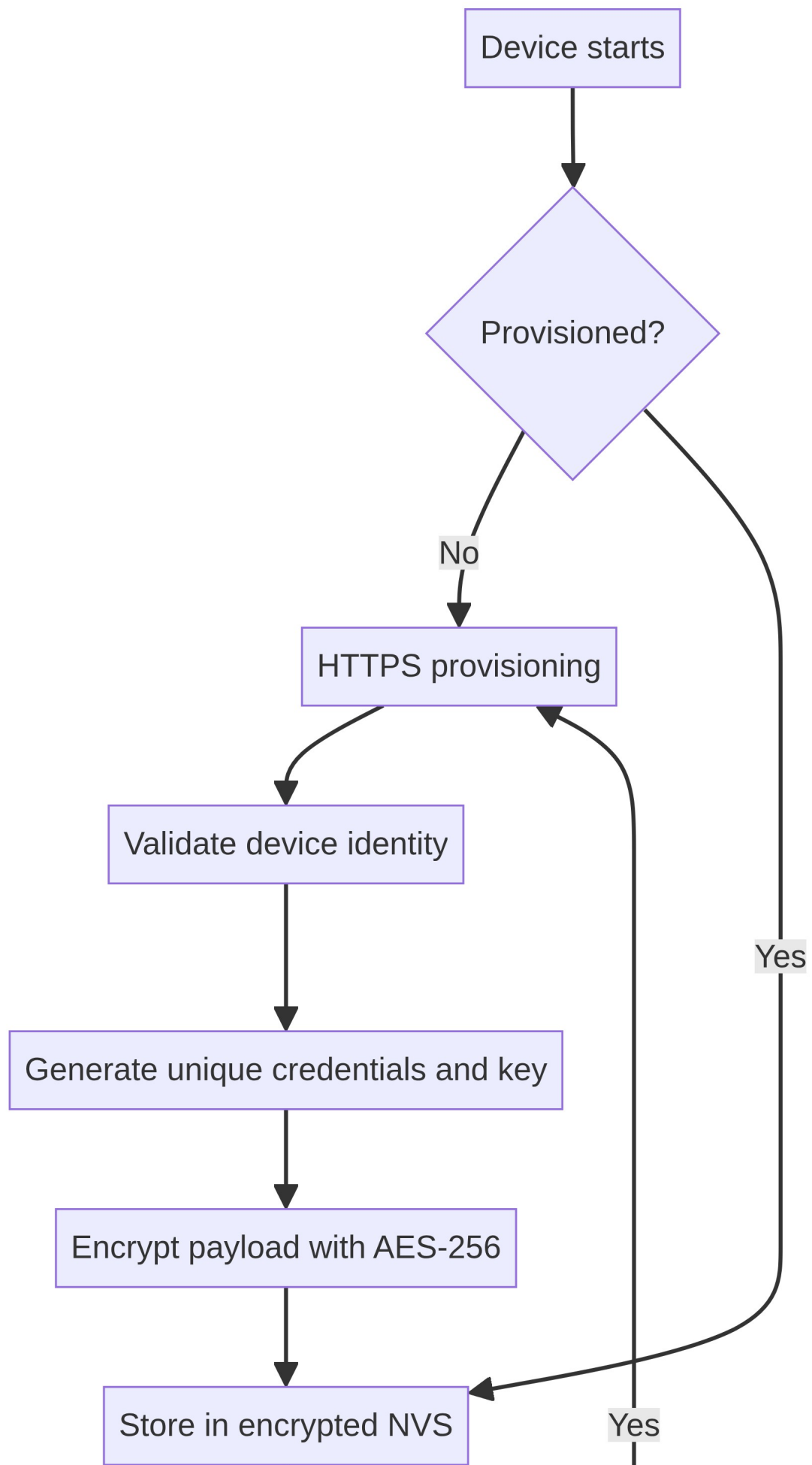
6.1 Development Environment

- Language: Python 3.14 for server and administration services; embedded firmware for ESP32.
- Framework: Flask 3.x for HTTPS provisioning and dashboard APIs.
- Broker: Mosquitto 2.x with MQTT over TLS.
- Database: SQLite 3.x.
- Frontend: Bootstrap 5.3.2 and Chart.js 4.4.0.
- Messaging library: paho-mqtt for MQTT client communication.
- Cryptography: AES-256 for sensitive values; Fernet-protected master key stored separately from the database.

6.2 Software Architecture

The software is divided into device, broker, application, data, and presentation layers. The device layer manages NVS, TLS, and sensor publication. The broker layer handles MQTT transport. The application layer contains `mqtt_service.py`, the provisioning API, `dashboard.py`, and administrative CLI functions. The data layer stores devices, permissions, sessions, audit logs, sensor readings, device metrics, credentials, device keys, and provisioning tokens/events. The dashboard presents monitoring and management functions through a REST-enabled interface.

6.3 Flowchart / State Diagram



6.4 Algorithms

The main security logic is lifecycle-oriented rather than a numerical control algorithm. A device is assigned a unique credential set; sensitive payloads are encrypted with AES-256; transport is protected with TLS; sessions are authenticated and audited; and credentials can be rotated or revoked. Authorization decisions are based on device status, permissions, and session validity. Brute-force protection limits repeated authentication failures, while replay resistance depends on fresh session and provisioning values.

7. System Integration

The ESP32 communicates with the Mosquitto broker using MQTT over TLS on port 8883. Initial provisioning is performed through HTTPS on port 5443. The dashboard is exposed on port 5000, while port 1883 is reserved for development and is documented as disabled in the secure configuration. The server integrates authentication, authorization, key management, provisioning, audit logging, and SQLite persistence. Timing is affected by Wi-Fi latency, TLS handshakes, broker availability, and embedded memory; therefore, provisioning and routine publication are kept as separate operational phases.

8. Testing and Validation

8.1 Testing Strategy

Validation combines unit checks for credential and encryption functions, integration tests for API/database/broker interaction, device tests for NVS and TLS connectivity, security tests for invalid credentials and repeated failures, and operational tests for dashboard actions. The source documentation reports 28 successful tests. Production acceptance should additionally include long-duration, fault-injection, recovery, and multi-device load testing.

8.2 Test Cases

Test Case ID	Input / Condition	Expected Output	Actual Output	Status
TC-01	New ESP32 requests provisioning	Unique encrypted credential package is issued once	Provisioning workflow documented as successful	Pass
TC-02	Device reconnects after provisioning	TLS connection and MQTT session are established	TLS/MQTT architecture configured on port 8883	Pass
TC-03	Database inspected without master key	Secrets remain unreadable	AES-256 encrypted values and separate master.key	Pass
TC-04	Invalid credentials submitted repeatedly	Request is rejected and logged	Brute-force protection and audit logging enabled	Pass
TC-05	Administrator rotates a device key	Old key is invalidated and new key is issued	Key rotation supported by lifecycle design	Pass
TC-06	Administrator revokes a device	Device cannot establish an authorized session	Suspend/revoke operations documented	Pass
TC-07	Dashboard monitoring opened	Device and session metrics are visible	Ten dashboard pages documented	Pass
TC-08	Development MQTT port checked	Insecure port is disabled in secure deployment	Port 1883 marked disabled for development	Pass

8.3 Hardware and Software Testing

Hardware troubleshooting should include regulated-voltage checks, continuity checks, Wi-Fi signal verification, and inspection of boot and NVS behavior. Software debugging should include API request logs, broker logs, audit records, database integrity checks, TLS certificate validation, and controlled invalid-input tests. Test evidence should record timestamps, device identifiers, software versions, and the expected versus observed result.

9. Results and Discussion

The documented implementation achieves the principal security goals: device-specific secrets, encrypted storage, TLS-protected communication, centralized provisioning, lifecycle operations, audit logging, and administrative monitoring. The reported project metrics are approximately 6,500 lines of code, more than 30 files, 11 database tables, four active services, ten dashboard pages, 28 successful tests, a security score of 9.5/10, and 98% completion. These results indicate a functional prototype with broad coverage. The main remaining validation gap is the absence of quantitative performance measurements under a large device fleet and long-duration fault conditions.

10. Challenges and Solutions

- CHALLENGE: Hardcoded credentials in firmware
- SOLUTION: Moved secrets to a centralized provisioning workflow and generated device-specific values.
- CHALLENGE: Database theft exposing secrets
- SOLUTION: Encrypted sensitive values with AES-256 and kept the Fernet master key separate from the database.
- CHALLENGE: One shared key for all devices
- SOLUTION: Introduced per-device credentials and key lifecycle operations.
- CHALLENGE: No recovery after leakage
- SOLUTION: Added rotation, suspension, revocation, audit logging, and dashboard administration.
- CHALLENGE: Constrained ESP32 resources
- SOLUTION: Used encrypted NVS and separated one-time provisioning from routine MQTT publication.
- CHALLENGE: Secure transport across multiple services
- SOLUTION: Standardized TLS 1.2+ for MQTT and HTTPS provisioning while disabling the development MQTT port in secure deployment.

11. Conclusion

The project delivers a coherent security architecture for managing credentials and keys across IoT devices. It replaces insecure hardcoded and shared secrets with unique provisioning, encrypted storage, TLS communication, authorization, auditability, and revocation. Based on the documented results, the initial objectives were substantially met. The system is suitable as a secure prototype and foundation for further hardening, performance testing, and deployment-scale validation.

12. Future Work

- Integrate a hardware secure element or Trusted Platform Module for stronger key protection.
- Add certificate-based device identity and automated certificate renewal.
- Perform large-fleet load testing and quantify provisioning and MQTT latency.
- Add high-availability deployment, database backup, and disaster recovery.
- Introduce formal threat modeling, penetration testing, and security audit evidence.
- Improve offline recovery, device factory reset, and secure firmware-update workflows.

13. References

1. Mosquitto Project, "Eclipse Mosquitto Documentation," technical documentation.
2. Python Software Foundation, "Python Documentation," technical documentation.
3. Pallets Projects, "Flask Documentation," technical documentation.
4. SQLite Consortium, "SQLite Documentation," technical documentation.
5. Espressif Systems, "ESP32-WROOM-32 Datasheet and ESP-IDF Programming Guide."
6. OASIS, "MQTT Version 3.1.1 and MQTT Version 5.0 Specifications."
7. National Institute of Standards and Technology, "Recommendation for Key Management," NIST Secure Credential & Key Management System for IoT Devices — 9

Special Publication 800-57.

8. National Institute of Standards and Technology, “Guidelines for Managing the Security of Mobile Devices in the Enterprise,” NIST Special Publication 800-124.

14. Appendices

Appendix A: Source Code

The source documentation identifies the main application components as `mqtt_service.py`, `provisioning_api`, `dashboard.py`, and `admin_cli.py`. The repository URL was not included in the supplied file; it should be added here before submission:

REPOSITORY LINK: [To be provided by the project team]

Appendix B: Additional Schematics / Figures

The block diagram and software flow diagram included in Sections 5.3 and 6.3 are the primary figures reconstructed from the supplied documentation. A pin-level circuit schematic and additional deployment figures can be inserted here when available.